Universidad del Valle de Guatemala
Facultad de Ingeniería
POO
Segundo Ciclo, 2026



Ejercicio 5

Antonio Molina - 26878

1. Requisitos funcionales

Información de los módulos

1. El sistema debe permitir administrar los módulos de la misión del satélite Quetzal-2.
2. Cada módulo debe guardar un ID, nombre, estado de salud, disponibilidad para operar y costo de construcción.
3. El sistema debe manejar tres tipos de módulos: vuelo, tierra y energía.
4. Los módulos de vuelo deben guardar el tipo de instrumento, que puede ser cámara o sensor.
5. Los módulos de vuelo deben guardar la cantidad de datos que recolectan por ciclo, la energía que consumen y sus ciclos acumulados.
6. Los módulos de vuelo deben guardar el total histórico de datos recolectados.
7. Los módulos de tierra deben guardar la capacidad de descarga, energía necesaria para descargar y total histórico descargado.
8. Los módulos de tierra deben guardar la estación a la que pertenecen, porque se necesita para el reporte de comunicaciones.
9. Los módulos de energía deben guardar la cantidad de energía que generan por ciclo y sus ciclos acumulados.
10. Los módulos de energía deben guardar el total histórico de energía generada.
11. Las capacidades de operación de los módulos deben mantenerse constantes durante la misión.
12. El programa debe iniciar con al menos 10 módulos de los tres tipos, almacenados en una única lista polimórfica.

Funcionamiento del simulador

13. El programa debe permitir avanzar los ciclos de la misión.
14. Cada módulo debe tener una oportunidad de operar durante cada ciclo.
15. Los módulos deben ejecutarse respetando el orden establecido en la carga inicial.
16. Los recursos deben actualizarse inmediatamente después de que cada módulo realice su operación.
17. La energía y los datos restantes deben conservarse entre ciclos.
18. Los módulos inactivos no deben realizar operaciones.
19. Un módulo de vuelo solamente puede recolectar datos si está activo y dispone de suficiente energía.
20. Los módulos de vuelo deben consumir toda la energía necesaria para recolectar sus datos.
21. No se deben permitir recolecciones parciales.
22. Los datos recolectados deben agregarse a los datos pendientes de descarga en órbita.
23. Los módulos de vuelo deben aumentar sus ciclos acumulados solamente cuando logran recolectar información.
24. Los módulos de tierra solamente pueden descargar datos si están activos, tienen suficiente energía y existen datos pendientes en órbita.
25. Los módulos de tierra no deben descargar más información que su capacidad por ciclo.
26. Los módulos de tierra tampoco pueden descargar más información que la disponible en órbita.
27. Cada descarga debe consumir toda la energía requerida, aunque se descarguen menos datos que la capacidad máxima.
28. Los datos descargados deben restarse de los datos pendientes en órbita.
29. Los datos descargados deben sumarse al total histórico de la misión y del módulo correspondiente.

30. Los módulos de energía activos deben generar una cantidad fija de energía por ciclo.
31. La energía generada debe quedar disponible inmediatamente para los módulos siguientes.
32. Los módulos de energía deben aumentar sus ciclos acumulados solamente cuando generan energía.
33. Los módulos inactivos no deben aumentar sus ciclos acumulados.
34. Si un módulo no puede operar, el programa debe mostrar el motivo y continuar con el siguiente.
35. El programa no debe implementar baterías.

Consultas y menú

36. El programa debe permitir consultar todos los módulos registrados y sus características.
37. El programa debe permitir buscar un módulo utilizando su ID.
38. El programa debe permitir buscar un módulo utilizando su nombre.
39. El programa debe permitir consultar los módulos ordenados por costo de construcción.
40. El catálogo ordenado no debe modificar el orden original de ejecución de los módulos.
41. El programa debe permitir avanzar un ciclo mediante una opción del menú.
42. Al finalizar un ciclo se deben mostrar los resultados de cada módulo.
43. Al finalizar un ciclo también se debe mostrar el estado de los recursos de la misión.
44. El sistema debe permitir consultar el resumen de comunicaciones.
45. El resumen debe mostrar la cantidad total de módulos de tierra.
46. El resumen debe mostrar cuántos módulos de tierra están activos.
47. El resumen debe mostrar la capacidad total de descarga de los módulos de tierra activos.
48. El resumen debe mostrar el total histórico de datos descargados.
49. El resumen debe identificar el módulo de tierra con mayor cantidad histórica de datos descargados.
50. Si varios módulos de tierra tienen la misma cantidad máxima descargada, deben mostrarse todos.
51. Para los módulos de tierra con mayor descarga deben mostrarse nombre, ID y estación.
52. El programa debe utilizar herencia para compartir los atributos y comportamientos comunes.
53. El programa debe utilizar polimorfismo para ejecutar el comportamiento particular de cada módulo.
54. El programa debe aplicar el patrón Modelo-Vista-Controlador.
55. El programa debe utilizar encapsulamiento y métodos públicos para acceder a la información necesaria.
56. El programa debe sobrescribir procesarCiclo() y toString() donde corresponda.
57. El menú debe mantenerse disponible hasta que el usuario decida salir.

Clases necesarias y propósito

Para desarrollar el programa se utilizarán nueve clases, organizadas siguiendo el patrón MVC.

Clase Modulo

La clase Modulo representa los atributos y comportamientos que tienen en común todos los módulos de la misión. Guarda el ID, nombre, salud, actividad y costo de construcción. También define el método procesarCiclo(), que cada clase hija deberá implementar según su función.

Clase ModuloVuelo

La clase ModuloVuelo hereda de Modulo y representa los instrumentos científicos del satélite. Se encarga de recolectar información cuando está activo y dispone de energía suficiente. También guarda los datos recolectados y la cantidad de ciclos en los que logró operar.

Clase ModuloTierra

La clase ModuloTierra hereda de Modulo y representa las antenas que reciben información del satélite. Su responsabilidad es descargar los datos disponibles en órbita, consumir la energía correspondiente y guardar el total histórico descargado. También aporta su información al resumen de comunicaciones.

Clase ModuloEnergia

La clase ModuloEnergia hereda de Modulo y representa los paneles solares.
Su responsabilidad es generar energía cuando se encuentra activo y registrar cuántos ciclos ha operado y cuánta energía ha producido.

Clase Mision

Administra la colección polimórfica de módulos, la energía disponible, los datos pendientes de descarga y los datos históricos descargados. También permite buscar módulos, consultar el catálogo y avanzar los ciclos.

Clase ResumenComunicaciones

Se encarga de reunir la información necesaria para el reporte de comunicaciones. Guarda la cantidad de módulos de tierra, los activos, su capacidad total de descarga, el total histórico descargado y los módulos que tienen la mayor cantidad de datos descargados.

Clase VistaConsola

Se encarga de la interacción con el usuario. Muestra el menú, solicita información y presenta los resultados. No realiza directamente los cálculos de energía ni de descarga de datos.

Clase ControladorMision

Recibe las opciones seleccionadas por el usuario y coordina las operaciones.
Utiliza Mision para consultar información y ejecutar los ciclos, y VistaConsola para mostrar los resultados.

Clase Main

Contiene el método main(), crea los objetos principales, registra los módulos iniciales y comienza la ejecución del controlador.

Atributos, tipos y propósito

Todos los atributos son privados

Clase Modulo

- id (String): guarda el identificador de cada módulo dentro de la misión. Se utiliza para encontrar un módulo específico.

- nombre (String): guarda el nombre del módulo para identificarlo y mostrar su información.
- salud (int): representa el estado de salud del módulo.
- activo (boolean): indica si el módulo se encuentra activo y puede realizar su operación durante los ciclos.
- costoConstruccion (double): guarda el costo de construcción del módulo y permite ordenar el catálogo.

Clase ModuloVuelo

- tipoInstrumento (String): indica si el módulo utiliza una cámara o un sensor.
- datosPorCiclo (double): guarda la cantidad de MB que puede recolectar durante cada ciclo.
- consumoEnergia (double): guarda la cantidad de energía necesaria para recolectar datos.
- ciclosAcumulados (int): guarda cuántos ciclos ha logrado recolectar información.
- totalRecolectado (double): acumula todos los MB recolectados durante la misión.

Clase ModuloTierra

- estacion (String): guarda el nombre de la estación terrestre a la que pertenece el módulo.
- capacidadDescarga (double): guarda la cantidad máxima de MB que puede descargar durante cada ciclo.
- energiaDescarga (double): representa la cantidad de energía necesaria para realizar una descarga.
- totalDescargado (double): guarda la cantidad histórica de datos descargados por el módulo.

Clase ModuloEnergia

- energiaPorCiclo (double): guarda la cantidad de energía que genera el módulo durante cada ciclo.
- ciclosAcumulados (int): registra la cantidad de ciclos en los que logró generar energía.
- totalGenerado (double): guarda la cantidad histórica de energía producida.

Clase Mision

- modulos (ArrayList<Modulo>): almacena todos los módulos de la misión en una sola colección polimórfica. Permite guardar objetos de las clases ModuloVuelo, ModuloTierra y ModuloEnergia.
- energiaDisponible (double): guarda la cantidad de energía que tiene disponible la misión para que los módulos puedan realizar sus operaciones.
- datosOrbita (double): guarda la cantidad de MB que han sido recolectados por los módulos de vuelo y todavía están pendientes de descarga.
- datosDescargados (double): guarda el total histórico de información que los módulos de tierra han descargado durante la misión.
- numeroCiclo (int): guarda la cantidad de ciclos que se han ejecutado en el simulador.

Clase ResumenComunicaciones

- totalModulosTierra (int): guarda la cantidad total de módulos de tierra registrados en la misión.
- modulosActivos (int): guarda cuántos módulos de tierra se encuentran activos.
- capacidadTotal (double): acumula la capacidad de descarga por ciclo de todos los módulos de tierra activos.

- totalDescargado (double): guarda la cantidad histórica de MB descargados por los módulos de tierra.
- mayorDescarga (double): guarda la mayor cantidad histórica de datos descargados por un módulo de tierra.
- modulosMayorDescarga (ArrayList<ModuloTierra>): almacena los módulos de tierra que comparten la mayor cantidad histórica de datos descargados.

Clase VistaConsola

- teclado (Scanner): permite leer las opciones y los datos ingresados por el usuario mediante la consola.

Clase ControladorMision

- mision (Mision): guarda la referencia al objeto que administra los módulos y los recursos de la misión.
- vista (VistaConsola): guarda la referencia al objeto que permite mostrar información y recibir las opciones del usuario.

Clase Main

Esta clase no necesita atributos de instancia porque su función es iniciar el programa y crear los objetos principales.

Métodos, parámetros y resultados

Los métodos de las clases serán públicos cuando necesiten utilizarse desde otras clases. Los métodos auxiliares que solamente se utilicen dentro de una misma clase podrán ser privados. Los constructores permitirán asignar los valores iniciales de cada objeto.

Clase Modulo

- Modulo(String id, String nombre, int salud, boolean activo, double costoConstruccion): constructor que recibe los datos comunes de un módulo y los utiliza para inicializar sus atributos. También permite que las clases hijas aprovechen estos datos mediante super().
- getId() : String: devuelve el identificador del módulo para poder buscarlo dentro de la misión.
- getNombre() : String: devuelve el nombre del módulo.
- getSalud() : int: devuelve el estado de salud del módulo.
- getActivo() : boolean: devuelve true si el módulo está activo y false si se encuentra inactivo.
- getCostoConstruccion() : double: devuelve el costo de construcción del módulo, que se utiliza para ordenar el catálogo.
- procesarCiclo(Mision mision) : String: método público y abstracto que recibe el objeto de la misión. Cada clase hija debe implementarlo para realizar su operación correspondiente. Devuelve un mensaje que indica qué ocurrió durante el ciclo.
- aportarResumenComunicaciones(ResumenComunicaciones resumen) : void: recibe el objeto encargado del reporte de comunicaciones. En la clase padre no realiza cambios, pero puede ser sobrescrito por las clases que necesiten aportar información.
- toString() : String: devuelve una cadena con la información general del módulo, como su ID, nombre, salud, actividad y costo de construcción.
- equals(Object obj) : boolean: compara dos módulos utilizando su ID. Devuelve true cuando representan el mismo módulo y false cuando son diferentes.

- () : int: genera un valor numérico a partir del ID del módulo, manteniendo la misma forma de identificarlo que utiliza equals().

Clase ModuloVuelo

- ModuloVuelo(String id, String nombre, int salud, boolean activo, double costoConstruccion, String tipoInstrumento, double datosPorCiclo, double consumoEnergia): constructor que recibe los atributos comunes y particulares del módulo de vuelo. Utiliza super() para inicializar los atributos heredados de Modulo.
- getTipoInstrumento() : String: devuelve el tipo de instrumento, que puede ser cámara o sensor.
- getDatosPorCiclo() : double: devuelve la cantidad de MB que el módulo puede recolectar durante un ciclo.
- getConsumoEnergia() : double: devuelve la cantidad de energía necesaria para recolectar información.
- getCiclosAcumulados() : int: devuelve la cantidad de ciclos en los que el módulo logró recolectar datos.
- getTotalRecolectado() : double: devuelve la cantidad histórica de MB recolectados por el módulo.
- procesarCiclo(Mision mision) : String: sobrescribe el método de la clase padre. Comprueba que el módulo esté activo y que la misión tenga suficiente energía. Si ambas condiciones se cumplen, consume la energía necesaria, agrega los datos recolectados a los pendientes en órbita y aumenta los ciclos acumulados. Si no puede operar, devuelve un mensaje indicando el motivo sin modificar los recursos.
- toString() : String: sobrescribe el método de la clase padre y devuelve los datos comunes del módulo junto con su tipo de instrumento, capacidad de recolección, consumo de energía, ciclos acumulados y total recolectado.

Clase ModuloTierra

- ModuloTierra(String id, String nombre, int salud, boolean activo, double costoConstruccion, String estacion, double capacidadDescarga, double energiaDescarga): constructor que inicializa los atributos comunes mediante super() y después guarda los datos particulares del módulo de tierra.
- getEstacion() : String: devuelve el nombre de la estación terrestre a la que pertenece el módulo.
- getCapacidadDescarga() : double: devuelve la cantidad máxima de MB que el módulo puede descargar durante un ciclo.
- getEnergiaDescarga() : double: devuelve la cantidad de energía necesaria para realizar una descarga.
- getTotalDescargado() : double: devuelve la cantidad histórica de información descargada por el módulo.
- procesarCiclo(Mision mision) : String: sobrescribe el método de la clase padre. Verifica que el módulo esté activo, que exista suficiente energía y que haya datos pendientes en órbita. Si puede operar, consume la energía completa de una descarga y recibe la cantidad de datos permitida por su capacidad y los MB disponibles. Después actualiza su total histórico y devuelve un mensaje con el resultado. Si alguna condición no se cumple, no modifica los recursos.

- aportarResumenComunicaciones(ResumenComunicaciones resumen) : void: sobrescribe el método de Modulo y envía la información del módulo de tierra al objeto encargado del reporte, para que pueda actualizar los contadores y los datos históricos.
- toString() : String: devuelve la información común y particular del módulo de tierra, incluyendo su estación, capacidad de descarga, consumo de energía y total descargado.

Clase ModuloEnergia

- ModuloEnergia(String id, String nombre, int salud, boolean activo, double costoConstruccion, double energiaPorCiclo): constructor que recibe los datos comunes y la cantidad de energía generada por ciclo. Utiliza super() para inicializar los atributos heredados.
- getEnergiaPorCiclo() : double: devuelve la cantidad de energía que produce el módulo durante cada ciclo.
- getCiclosAcumulados() : int: devuelve la cantidad de ciclos en los que el módulo generó energía.
- getTotalGenerado() : double: devuelve la cantidad histórica de energía producida.
- procesarCiclo(Mision mision) : String: sobrescribe el método de la clase padre. Verifica si el módulo está activo y, si puede operar, agrega la energía correspondiente a la misión, aumenta sus ciclos acumulados y actualiza el total histórico generado. Si está inactivo, no produce energía y devuelve un mensaje explicando la situación.
- toString() : String: devuelve los datos comunes y particulares del módulo de energía, incluyendo su capacidad de generación, ciclos acumulados y total histórico producido.

Clase Mision

- Mision(double energiaInicial): constructor que recibe la cantidad inicial de energía disponible. También crea el ArrayList de módulos e inicializa los datos en órbita, los datos descargados y el contador de ciclos en cero.
- agregarModulo(Modulo modulo) : boolean: recibe un objeto de tipo Modulo y verifica que su ID no esté repetido. Si puede registrarlo, lo agrega a la colección y devuelve true. Si el ID ya existe, devuelve false.
- buscarPorId(String id) : Modulo: recibe el ID de un módulo y recorre la colección para encontrarlo. Devuelve el objeto correspondiente o null si no existe.
- buscarPorNombre(String nombre) : Modulo: recibe el nombre de un módulo y busca una coincidencia dentro de la colección. Devuelve el primer módulo encontrado o null si no existe.
- getModulos() : ArrayList de Modulo: devuelve la colección de módulos registrados en la misión.
- obtenerCatalogoOrdenado() : ArrayList de Modulo: crea una copia de la colección y la ordena de menor a mayor costo de construcción. Devuelve la lista ordenada sin modificar el orden original de los módulos.
- avanzarCiclo() : ArrayList de String: aumenta el contador de ciclos y recorre los módulos respetando su orden original. Cada módulo ejecuta su propio método

procesarCiclo() y devuelve un mensaje. Todos los mensajes se almacenan en una colección que se devuelve al controlador.
- getEnergiaDisponible() : double: devuelve la cantidad actual de energía disponible.
- getDatosOrbita() : double: devuelve la cantidad de MB recolectados que todavía están pendientes de descarga.
- getDatosDescargados() : double: devuelve la cantidad histórica de MB descargados durante la misión.
- getNumeroCiclo() : int: devuelve la cantidad de ciclos ejecutados.
- agregarEnergia(double cantidad) : void: recibe una cantidad de energía y la agrega a los recursos disponibles de la misión.
- consumirEnergia(double cantidad) : boolean: recibe la cantidad de energía necesaria para una operación. Verifica si existe suficiente energía y, si es posible, la descuenta y devuelve true. Si no alcanza, devuelve false sin modificar los recursos.
- agregarDatos(double cantidad) : void: recibe una cantidad de MB recolectados y los agrega a los datos pendientes en órbita.
- descargarDatos(double cantidad) : void: recibe una cantidad de MB descargados, los resta de los datos pendientes en órbita y los suma al total histórico descargado por la misión.
- generarResumenComunicaciones() : ResumenComunicaciones: crea un objeto encargado del reporte y recorre los módulos para solicitarles que aporten su información. Finalmente devuelve el resumen con los datos recopilados.

Clase ResumenComunicaciones
- ResumenComunicaciones(): constructor que inicializa en cero los contadores de módulos de tierra, módulos activos, capacidad total de descarga, datos históricos descargados y mayor descarga. También crea un ArrayList vacío para guardar los módulos de tierra que tengan la mayor cantidad de datos descargados.
- agregarModuloTierra(ModuloTierra modulo) : void: recibe un objeto de tipo ModuloTierra y utiliza su información para actualizar el resumen. Aumenta la cantidad total de módulos de tierra y, si el módulo está activo, aumenta la cantidad de módulos activos y suma su capacidad de descarga. También agrega sus datos históricos al total descargado y verifica si tiene la mayor cantidad de información descargada. Si varios módulos comparten la cantidad máxima, los guarda a todos en la colección.
- getTotalModulosTierra() : int: devuelve la cantidad total de módulos de tierra registrados en la misión, independientemente de si están activos o inactivos.
- getModulosActivos() : int: devuelve la cantidad de módulos de tierra que se encuentran activos.
- getCapacidadTotal() : double: devuelve la suma de las capacidades de descarga por ciclo de todos los módulos de tierra activos.
- getTotalDescargado() : double: devuelve la cantidad histórica total de MB descargados por los módulos de tierra durante la misión.
- getMayorDescarga() : double: devuelve la mayor cantidad histórica de MB descargados por un solo módulo de tierra.
- getModulosMayorDescarga() : ArrayList de ModuloTierra: devuelve la colección de módulos de tierra que comparten la mayor cantidad histórica descargada. Esto permite mostrar varios módulos cuando existe un empate.

Clase VistaConsola

- VistaConsola(): constructor que inicializa el objeto Scanner para leer los datos ingresados por el usuario desde la consola.
- mostrarMenu() : void: muestra las opciones disponibles del programa, como listar módulos, buscar un módulo, consultar el catálogo, avanzar un ciclo, consultar el resumen de comunicaciones y salir.
- leerOpcion() : int: solicita al usuario que ingrese una opción del menú y devuelve el número seleccionado para que el controlador pueda realizar la operación correspondiente.
- leerTexto(String mensaje) : String: recibe un mensaje que se muestra en pantalla para solicitar información al usuario. Después lee el texto ingresado y lo devuelve.
- mostrarMensaje(String mensaje) : void: recibe una cadena de texto y la muestra en la consola. Se utiliza para presentar mensajes de confirmación, errores y resultados de las operaciones.
- mostrarModulo(Modulo modulo) : void: recibe un objeto de tipo Modulo y muestra su información utilizando el método toString(). Gracias al polimorfismo, se muestran los datos correspondientes a cada categoría sin tener que verificar el tipo del objeto.
- mostrarRecursos(Mision mision) : void: recibe el objeto de la misión y muestra la cantidad de energía disponible, los datos pendientes en órbita y el total histórico descargado. Se utiliza principalmente después de ejecutar un ciclo.
- mostrarResumen(ResumenComunicaciones resumen) : void: recibe el resumen de comunicaciones y muestra la cantidad total de módulos de tierra, los activos, su capacidad total de descarga y los datos históricos descargados. También muestra el nombre, ID y estación del módulo con mayor descarga. Si varios módulos comparten el máximo, los muestra a todos.

Clase ControladorMision

- ControladorMision(Mision mision, VistaConsola vista): constructor que recibe el objeto que administra la misión y el objeto encargado de la interacción con el usuario. Guarda ambas referencias para coordinar las operaciones del programa.
- iniciar() : void: mantiene activo el menú principal hasta que el usuario seleccione la opción de salir. Recibe las opciones ingresadas mediante la vista y llama a los métodos correspondientes del controlador.
- listarModulos() : void: obtiene la colección de módulos registrados en Mision y utiliza los métodos de VistaConsola para mostrar la información de cada uno.
- buscarModulo() : void: permite seleccionar si se desea buscar un módulo por ID o por nombre. Solicita el dato mediante la vista, utiliza el método correspondiente de Mision y muestra la información del módulo encontrado. Si no existe, presenta un mensaje indicando que no se encontró.
- mostrarCatalogo() : void: solicita a Mision la colección de módulos ordenada por costo de construcción y utiliza la vista para mostrarla. Esta operación no modifica el orden original en que participan los módulos durante los ciclos.
- avanzarCiclo() : void: solicita a Mision ejecutar un ciclo completo. Recibe los mensajes de resultado de cada módulo y los muestra mediante la vista. Al finalizar también presenta los recursos disponibles de la misión.
- mostrarComunicaciones() : void: solicita a Mision generar el resumen de comunicaciones y envía el resultado a VistaConsola para mostrarlo al usuario.

Clase Main

- main(String[] args) : void: es el método principal donde comienza la ejecución del programa. Crea el objeto Mision con los recursos iniciales y registra al menos diez módulos de las tres categorías. Después crea los objetos VistaConsola y ControladorMision, y utiliza el método iniciar() del controlador para comenzar el menú principal.

Relaciones entre las clases

- ModuloVuelo, ModuloTierra y ModuloEnergia heredarán de Modulo.
- Mision administrará una colección de objetos Modulo.
- Mision utilizará ResumenComunicaciones para generar los reportes.
- ModuloTierra aportará información a ResumenComunicaciones.
- ControladorMision tendrá una relación con Mision y VistaConsola, porque necesita utilizar ambos objetos para coordinar las operaciones.
- Main creará los objetos principales y comenzará la ejecución del programa.

Aplicación del patrón MVC

- Modelo: estará formado por Modulo, ModuloVuelo, ModuloTierra, ModuloEnergia, Mision y ResumenComunicaciones. Estas clases guardan los datos y realizan las operaciones del simulador.
- Vista: estará formada por VistaConsola. Esta clase se encargará de mostrar información, solicitar opciones y presentar resultados.
- Controlador: estará formado por ControladorMision. Esta clase recibirá las opciones seleccionadas por el usuario y coordinará las operaciones entre el modelo y la vista.

.

Justificación del polimorfismo

El polimorfismo se utilizará principalmente mediante procesarCiclo(Mision mision). Este método será abstracto en Modulo y cada clase hija lo sobrescribirá utilizando @Override.

- En ModuloVuelo, el método permitirá recolectar datos científicos cuando exista suficiente energía.
- En ModuloTierra, permitirá descargar información pendiente desde órbita.
- En ModuloEnergia, permitirá generar energía para la misión.

Todos los módulos estarán guardados en una sola colección de tipo ArrayList<Modulo>. Esto permite recorrerlos y llamar al mismo método, aunque cada objeto realice una operación diferente.

Justificación de la herencia

La herencia se utilizará porque los tres tipos de módulos tienen características comunes. Todos necesitan guardar un ID, nombre, salud, estado de actividad y costo de construcción. Estos atributos estarán en la clase padre Modulo.

Las clases ModuloVuelo, ModuloTierra y ModuloEnergia heredarán de Modulo utilizando extends. Sus constructores utilizarán super() para inicializar los atributos comunes y después asignarán los valores particulares.

La clase Modulo será abstracta porque no se necesita crear módulos sin una función específica.

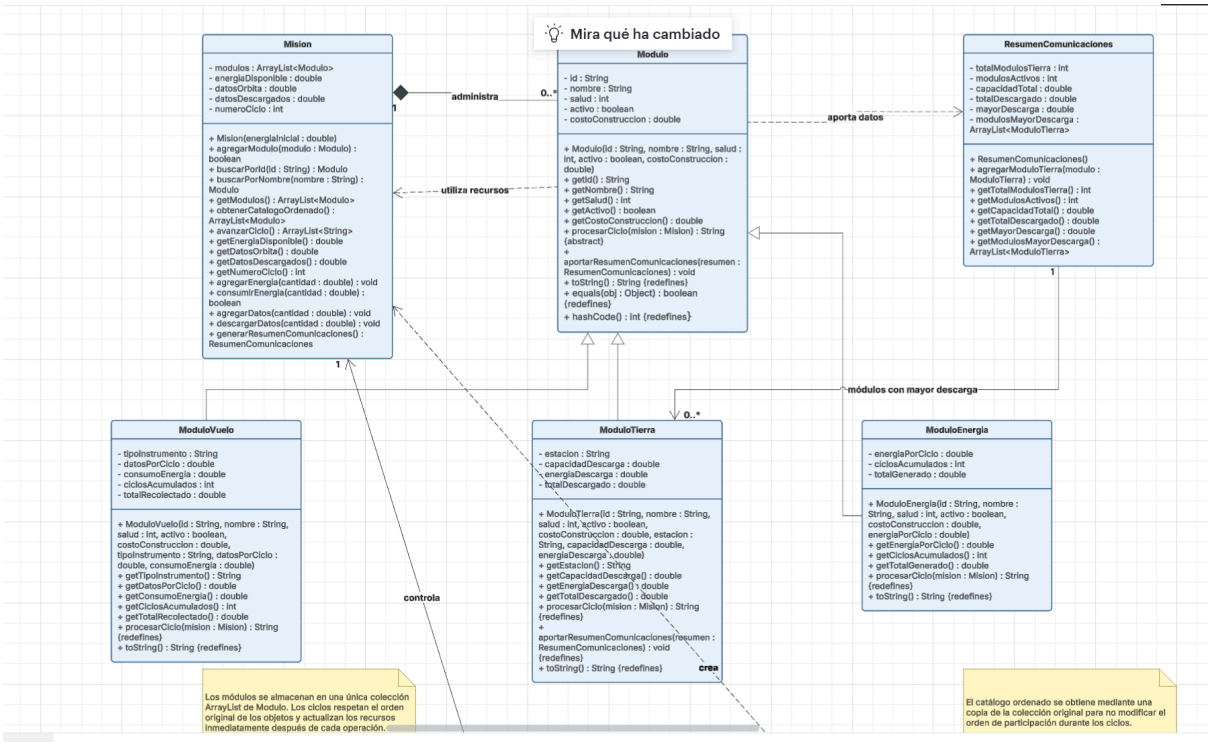
Valores iniciales de los objetos

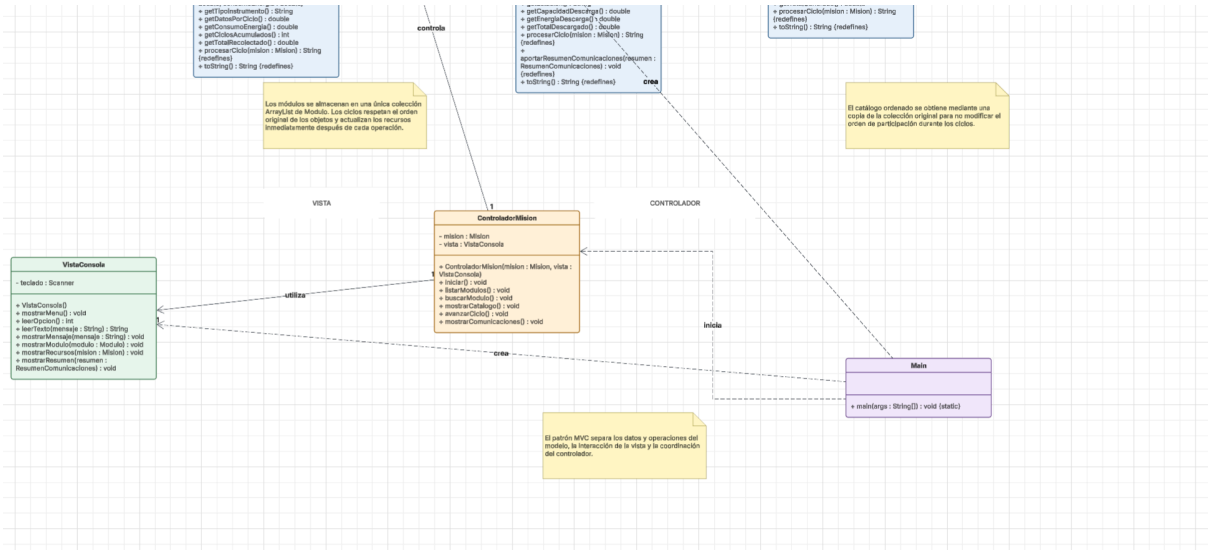
Al iniciar el programa se creará una misión con 10 unidades de energía disponible. Los datos pendientes en órbita, los datos descargados y el contador de ciclos comenzarán en cero. Se registrarán al menos diez módulos: cuatro de vuelo, tres de tierra y tres de energía. Cada uno tendrá un ID diferente, nombre, salud, estado de actividad, costo de construcción y sus características particulares.

- Los módulos de vuelo comenzarán con cero ciclos acumulados y cero MB históricos recolectados.
- Los módulos de tierra comenzarán con cero MB históricos descargados.
- Los módulos de energía comenzarán con cero ciclos acumulados y cero unidades históricas generadas.

Algunos módulos podrán comenzar inactivos para comprobar que no realizan operaciones. Los valores iniciales concretos se definirán en Main, manteniendo el orden en que deben participar durante los ciclos.

UML


Mira qué ha cambiado
Mision
- modulos : ArrayList<Modulo>
- energiaDisponible : double
- datosOrbita : double
- datosDescargados : double
- numeroCiclo : int
+ Mision(energiaInicial : double)
+ agregarModulo(modulo : Modulo) : boolean
+ buscarPorId(id : String) : Modulo
+ buscarPorNombre(nombre : String) : Modulo
+ getModulos() : ArrayList<Modulo>
+ obtenerCatalogoOrdenado() : ArrayList<Modulo>
+ avanzarCiclo() : ArrayList<String>
+ getEnergiaDisponible() : double
+ getDatosOrbita() : double
+ getDatosDescargados() : double
+ getNumeroCiclo() : int
+ agregarEnergia(cantidad : double) : void
+ consumirEnergia(cantidad : double) : boolean
+ agregarDatos(cantidad : double) : void
+ descargarDatos(cantidad : double) : void
+ generarResumenComunicaciones() : ResumenComunicaciones
administra
1
0..*
utiliza recursos
Modulo
- id : String
- nombre : String
- salud : int
- activo : boolean
- costoConstruccion : double
+ Modulo(id : String, nombre : String, salud : int, activo : boolean, costoConstruccion : double)
+ getId() : String
+ getNombre() : String
+ getSalud() : int
+ getActivo() : boolean
+ getCostoConstruccion() : double
+ procesarCiclo(mision : Mision) : String {abstract}
+ aportarResumenComunicaciones(resumen : ResumenComunicaciones) : void
+ toString() : String {redefines}
+ equals(obj : Object) : boolean {redefines}
+ hashCode() : int {redefines}
aporta datos
ResumenComunicaciones
- totalModulosTierra : int
- modulosActivos : int
- capacidadTotal : double
- totalDescargado : double
- mayorDescarga : double
- modulosMayorDescarga : ArrayList<ModuloTierra>
+ ResumenComunicaciones()
+ agregarModuloTierra(modulo : ModuloTierra) : void
+ getTotalModulosTierra() : int
+ getModulosActivos() : int
+ getCapacidadTotal() : double
+ getTotalDescargado() : double
+ getMayorDescarga() : double
+ getModulosMayorDescarga() : ArrayList<ModuloTierra>
1
módulos con mayor descarga
0..*
ModuloVuelo
- tipoInstrumento : String
- datosPorCiclo : double
- consumoEnergia : double
- ciclosAcumulados : int
- totalRecolectado : double
+ ModuloVuelo(id : String, nombre : String, salud : int, activo : boolean, costoConstruccion : double, tipoInstrumento : String, datosPorCiclo : double, consumoEnergia : double)
+ getTipoInstrumento() : String
+ getDatosPorCiclo() : double
+ getConsumoEnergia() : double
+ getCiclosAcumulados() : int
+ getTotalRecolectado() : double
+ procesarCiclo(mision : Mision) : String {redefines}
+ toString() : String {redefines}
controla
ModuloTierra
- estacion : String
- capacidadDescarga : double
- energiaDescarga : double
- totalDescargado : double
+ ModuloTierra(id : String, nombre : String, salud : int, activo : boolean, costoConstruccion : double, estacion : String, capacidadDescarga : double, energiaDescarga : double)
+ getEstacion() : String
+ getCapacidadDescarga() : double
+ getEnergiaDescarga() : double
+ getTotalDescargado() : double
+ procesarCiclo(mision : Mision) : String {redefines}
+ aportarResumenComunicaciones(resumen : ResumenComunicaciones) : void {redefines}
+ toString() : String {redefines}
crea
ModuloEnergia
- energiaPorCiclo : double
- ciclosAcumulados : int
- totalGenerado : double
+ ModuloEnergia(id : String, nombre : String, salud : int, activo : boolean, costoConstruccion : double, energiaPorCiclo : double)
+ getEnergiaPorCiclo() : double
+ getCiclosAcumulados() : int
+ getTotalGenerado() : double
+ procesarCiclo(mision : Mision) : String {redefines}
+ toString() : String {redefines}
Los módulos se almacenan en una única colección ArrayList de Modulo. Los ciclos respetan el orden original de los objetos y actualizan los recursos inmediatamente después de cada operación.
El catálogo ordenado se obtiene mediante una copia de la colección original para no modificar el orden de participación durante los ciclos.


controla
crea
Los módulos se almacenan en una única colección ArrayList de Modulo. Los ciclos respetan el orden original de los objetos y actualizan los recursos inmediatamente después de cada operación.
El catálogo ordenado se obtiene mediante una copia de la colección original para no modificar el orden de participación durante los ciclos.
VISTA
CONTROLADOR
ControladorMision
- mision : Mision
- vista : VistaConsola
+ ControladorMision(mision : Mision, vista : VistaConsola)
+ iniciar() : void
+ listarModulos() : void
+ buscarModulo() : void
+ mostrarCatalogo() : void
+ avanzarCiclo() : void
+ mostrarComunicaciones() : void
VistaConsola
- teclado : Scanner
+ VistaConsola()
+ mostrarMenu() : void
+ leerOpcion() : int
+ leerTexto(mensaje : String) : String
+ mostrarMensaje(mensaje : String) : void
+ mostrarModulo(modulo : Modulo) : void
+ mostrarRecursos(mision : Mision) : void
+ mostrarResumen(resumen : ResumenComunicaciones) : void
utiliza
inicia
Main
+ main(args : String[]) : void {static}
El patrón MVC separa los datos y operaciones del modelo, la interacción de la vista y la coordinación del controlador.